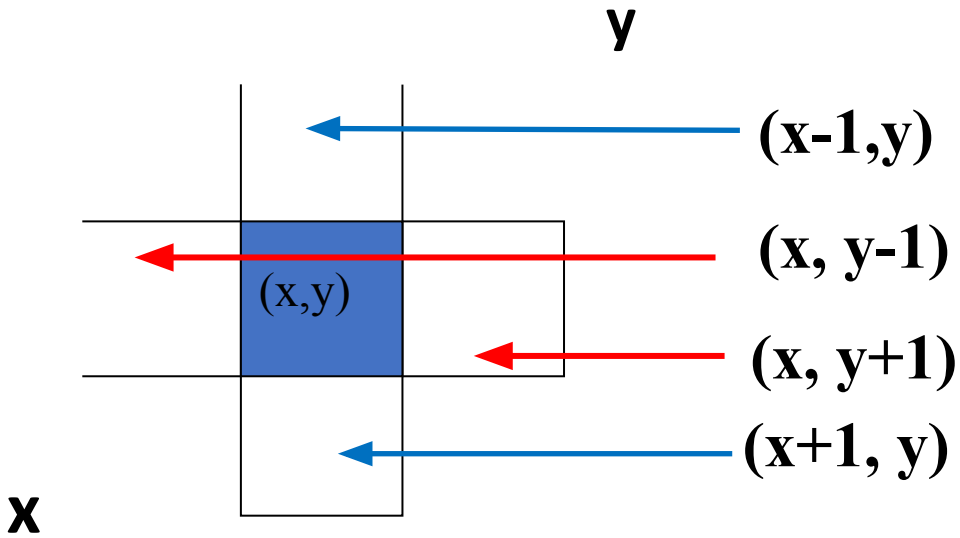
# MODULE 3

Neighbour of pixels, Connectivity, Relations, Equivalence & Transitive Closure; Distance Measures, Arithmetic Operations

# NEIGHBOURS OF A PIXEL

## A. 4 NEIGHBOURS

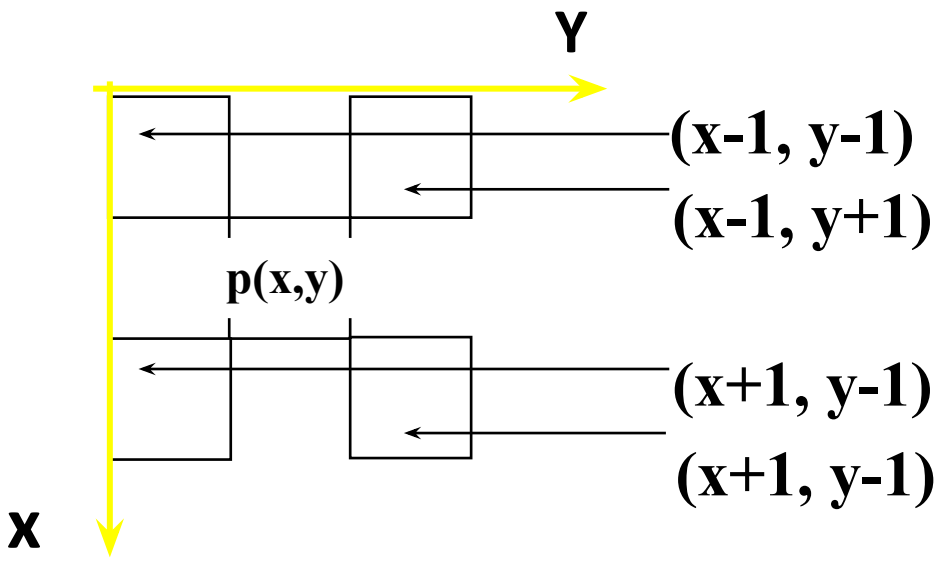
A pixel p at coordinates (x,y) has 4 **horizontal and vertical neighbours**, each being a unit distance from (x,y).




Each pixel is one unit distance from (x,y) and some of the neighbors of p lie outside the digital image if (x,y) is on the border of the image.

# B) Diagonal Neighbours $N_D(p)$

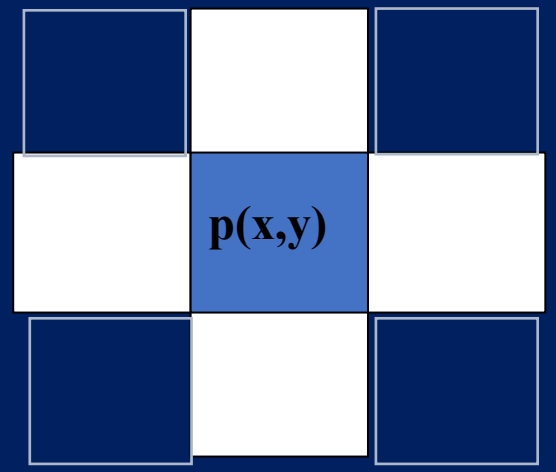
- A pixel p at coordinates (x,y) has 4 diagonal neighbours, each being a **unit distance** from (x,y)



# C) 8 Neighbours $N_8(p)$

8-neighbours of p= 4 diagonal neighbours of p and 4-neighbours of p

$$N_8(p) = N_D(p) + N_4(p)$$



# Adjacency and Connectivity

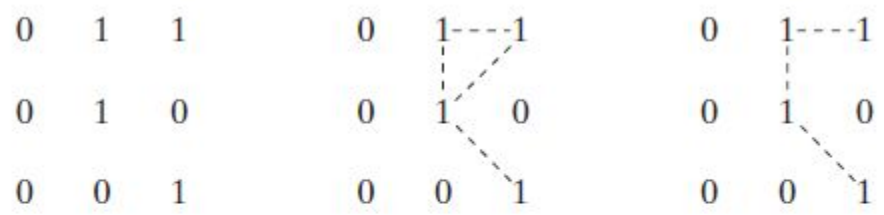
- Let $V$: a set of intensity values used to define adjacency and connectivity.
- In a binary image, $V = \{1\}$, if we are referring to adjacency of pixels with value 1.
- In a gray-scale image, the idea is the same, but $V$ typically contains more elements, for example, $V = \{180, 181, 182, \ldots, 200\}$
- If the possible intensity values 0 – 255, $V$ set can be any subset of these 256 values.

# Types of Adjacency

1. **4-adjacency:** Two pixels *p* and *q* with values from *V* are 4-adjacent if *q* is in the set $N_4(p)$.
2. **8-adjacency:** Two pixels *p* and *q* with values from *V* are 8-adjacent if *q* is in the set $N_8(p)$.
3. **m-adjacency =(mixed)**

# Types of Adjacency

- Mixed adjacency is a modification of 8-adjacency. It is introduced to eliminate the ambiguities that often arise when 8-adjacency is used.
- For example:




a b c

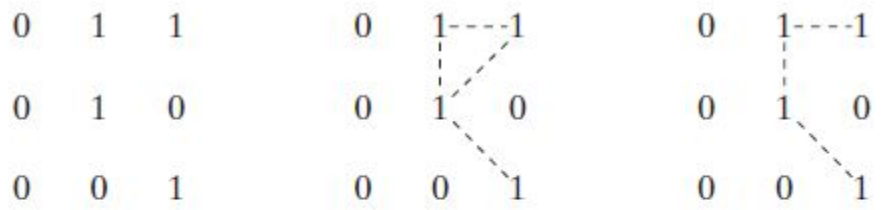
**FIGURE 2.26** (a) Arrangement of pixels; (b) pixels that are 8-adjacent (shown dashed) to the center pixel; (c) *m*-adjacency.

# A Digital Path

- A digital path (or curve) from pixel $p$ with coordinate $(x,y)$ to pixel $q$ with coordinate $(s,t)$ is a sequence of distinct pixels with coordinates $(x_0,y_0)$, $(x_1,y_1)$, …, $(x_n, y_n)$ where $(x_0,y_0) = (x,y)$ and $(x_n, y_n) = (s,t)$ and pixels $(x_i, y_i)$ and $(x_{i-1}, y_{i-1})$ are adjacent for $1 \leq i \leq n$
- n is the length of the path
- If $(x_0,y_0) = (x_n, y_n)$, the path is closed.
- We can specify 4-, 8- or m-paths depending on the type of adjacency specified.

# A Digital Path

- Return to the previous example:




a b c

**FIGURE 2.26** (a) Arrangement of pixels; (b) pixels that are 8-adjacent (shown dashed) to the center pixel; (c) *m*-adjacency.

In figure (b) the paths between the top right and bottom right pixels are 8-paths. And the path between the same 2 pixels in figure (c) is m-path

# Connectivity

- Connectivity refers to how pixels are connected to each other in an image based on a predefined neighborhood relationship. It plays a crucial role in image segmentation, object detection, and morphological processing.
- Let $S$ represent a subset of pixels in an image, two pixels $p$ and $q$ are said to be connected in $S$ if there exists a path between them consisting entirely of pixels in $S$.
- For any pixel $p$ in $S$, the set of pixels that are connected to it in $S$ is called a *connected component* of $S$. If it only has one connected component, then set $S$ is called a *connected set*.

# Region and Boundary

- **Region**

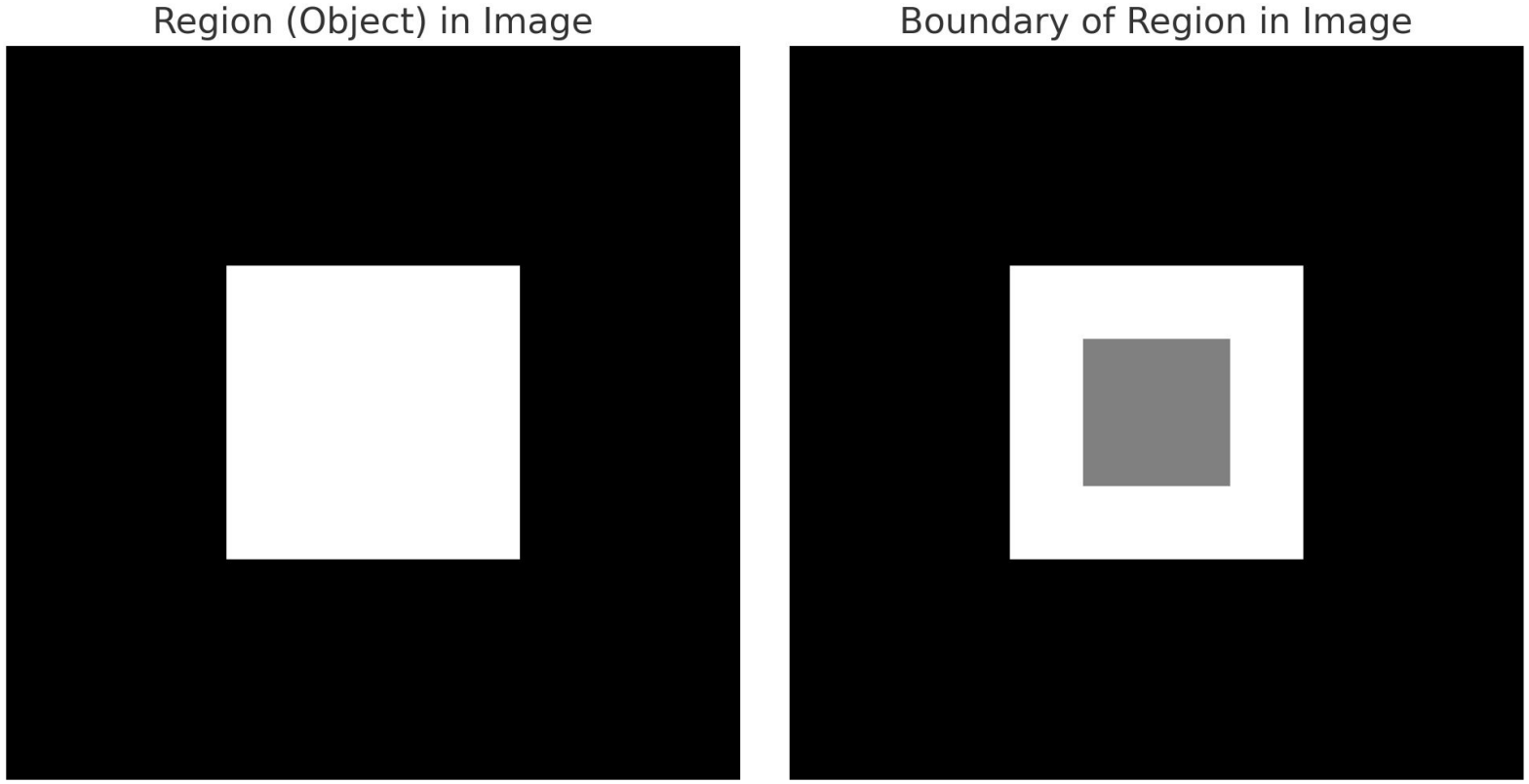
  Let *R* be  a subset of pixels in an image, we call *R* a region of the image if *R* is a connected set.

- **Boundary**

The *boundary* (also called *border* or *contour*) of a region *R* is the set of pixels in the region that have one or more neighbors that are not in *R*.

# Region and Boundary

If *R* happens to be an entire image, then its boundary is defined as the set of pixels in the first and last rows and columns in the image.



# Distance Measures

- In image processing and computer vision, **distance metrics** are used to measure similarity, clustering, and edge detection.
- For pixels $p$, $q$ and $z$, with coordinates $(x,y)$, $(s,t)$ and $(v,w)$, respectively, $D$ is a distance function if:

  (a) $D(p,q) \geq 0$ ($D(p,q) = 0$ iff $p = q$),

  (b) $D(p,q) = D(q, p)$, and

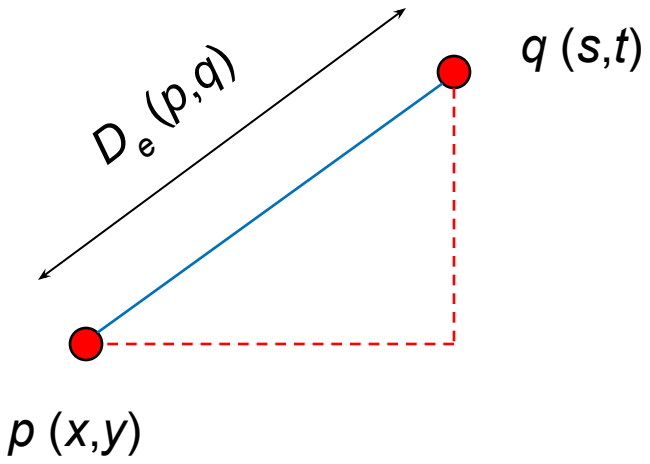
  (c) $D(p,z) \leq D(p,q) + D(q,z)$.

# Distance Measures

- The *Euclidean Distance* between *p* and *q* is defined as:

$$D_e(p,q) = [(x - s)^2 + (y - t)^2]^{1/2}$$

Pixels having a distance less than or equal to some value r from (x,y) are the points contained in a disk of radius r centered at (x,y)

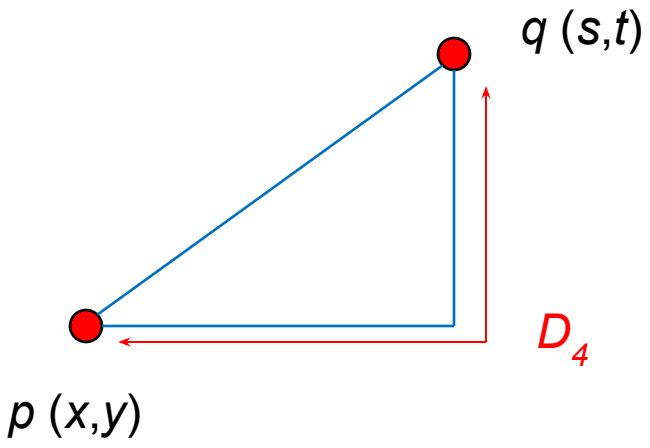


# Distance Measures

- The $D_4$ *distance* (also called *city-block distance*) between $p$ and $q$ is defined as:

$$D_4(p,q) = |x - s| + |y - t|$$

Pixels having a $D_4$ distance from (x,y), less than or equal to some value r form a Diamond centered at (x,y)



# Distance Measures

Example:

The pixels with distance $D_4 \leq 2$ from $(x,y)$ form the following contours of constant distance.

The pixels with $D_4 = 1$ are the 4-neighbors of (x,y)

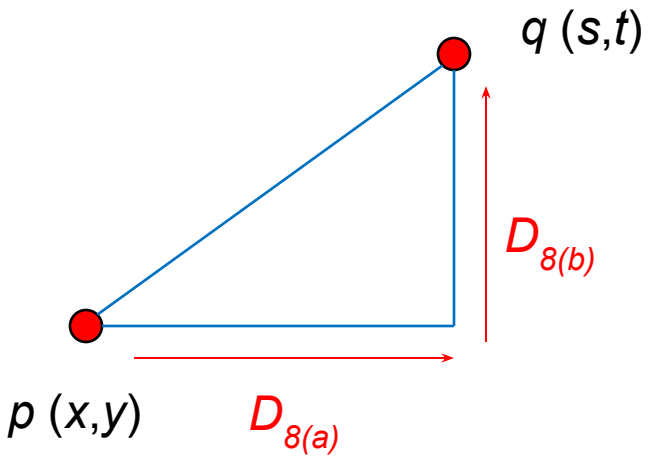
| | | | | |
|---|---|---|---|---|
| | | 2 | | |
| | 2 | 1 | 2 | |
| 2 | 1 | 0 | 1 | 2 |
| | 2 | 1 | 2 | |
| | | 2 | | |

# Distance Measures

- The $D_8$ *distance* (also called c*hessboard distance*) between $p$ and $q$ is defined as:

$$D_8(p,q) = \max(|x-s|, |y-t|)$$

Pixels having a $D_8$ distance from (x,y), less than or equal to some value r form a square Centered at (x,y)




$$D_8 = \max(D_{8(a)}, D_{8(b)})$$

# Distance Measures

Example:

$D_8$ distance ≤ 2 from (x,y) form the following contours of constant distance.

| | | | | |
|---|---|---|---|---|
| 2 | 2 | 2 | 2 | 2 |
| 2 | 1 | 1 | 1 | 2 |
| 2 | 1 | 0 | 1 | 2 |
| 2 | 1 | 1 | 1 | 2 |
| 2 | 2 | 2 | 2 | 2 |

# Distance Measures

- **Dm distance:**

  is defined as the shortest m-path between the points.

  In this case, the distance between two pixels will depend on the values of the pixels along the path, as well as the values of their neighbors.

# Distance Measures

- Example:

  Consider the following arrangement of pixels and assume that $p$, $p_2$, and $p_4$ have value 1 and that $p_1$ and $p_3$ can have can have a value of 0 or 1

  Suppose that we consider

  the adjacency of pixels

  values 1 (i.e. $V = \{1\}$)

| | | |
|---|---|---|
| | $p_3$ | $p_4$ |
| $p_1$ | $p_2$ | |
| $p$ | | |

# Distance Measures

- Cont. Example:

  Now, to compute the $D_m$ between points $p$ and $p_4$

  Here we have 4 cases:

  **Case1:** If $p_1$ =0 and $p_3$ = 0

  The length of the shortest m-path

  (the $D_m$ *distance) is 2* $(p, p_2, p_4)$

| | | |
|---|---|---|
| | 0 | 1 |
| 0 | 1 | |
| 1 | | |

# Distance Measures

- Cont. Example:

  **Case2:** If $p_1$ =1 and $p_3$ = 0

  now, $p_1$ *and p will no longer be adjacent (see m-adjacency definition)*

  then, the length of the shortest

  path will be 3 ($p$, $p_1$, $p_2$, $p_4$)

| | | |
|---|---|---|
| | 0 | 1 |
| 1 | 1 | |
| 1 | | |

# Distance Measures

- Cont. Example:

  **Case3:** If $p_1$ =0 and $p_3$ = 1

  The same applies here, and the shortest –m-path will be 3 ($p$, $p_2$, $p_3$, $p_4$)

| | | |
|---|---|---|
| | 1 | 1 |
| 0 | 1 | |
| 1 | | |

# Distance Measures

- Cont. Example:

  **Case4:** If $p_1$ =1 and $p_3$ = 1

  The length of the shortest m-path will be 4 ($p$, $p_1$ , $p_2$, $p_3$, $p_4$)

| | | |
|---|---|---|
| | 1 | 1 |
| 1 | 1 | |
| 1 | | |

# Arithmetic Operations

Arithmetic and logical operations between pixels are used in most branches of image processing. Arithmetic operations apply to multivalued pixels. Logic operations apply only to binary images.

Arithmetic operations between two images $f(x,y)$ and $g(x,y)$ are denoted as

$$\begin{aligned} s(x,y) &= f(x,y) + g(x,y) \\ d(x,y) &= f(x,y) - g(x,y) \\ p(x,y) &= f(x,y) \times g(x,y) \\ v(x,y) &= f(x,y) \div g(x,y) \end{aligned}$$

These are elementwise operations means that they are performed between corresponding pixel pairs in $f$ and $g$ for $x = 0,1,2,\ldots,M-1$ and $y = 0,1,2,\ldots,N-1$. As usual, $M$ and $N$ are the row and column sizes of the images. Clearly, $s, d, p$, and $v$ are images of size $M \times N$ also. Note that image arithmetic in the manner just defined involves images of the same size. The following examples illustrate the important role of arithmetic operations in digital image processing.

For entire images, Arithmetic operations are carried out pixel by pixel.

Example:

- Consider the matrices

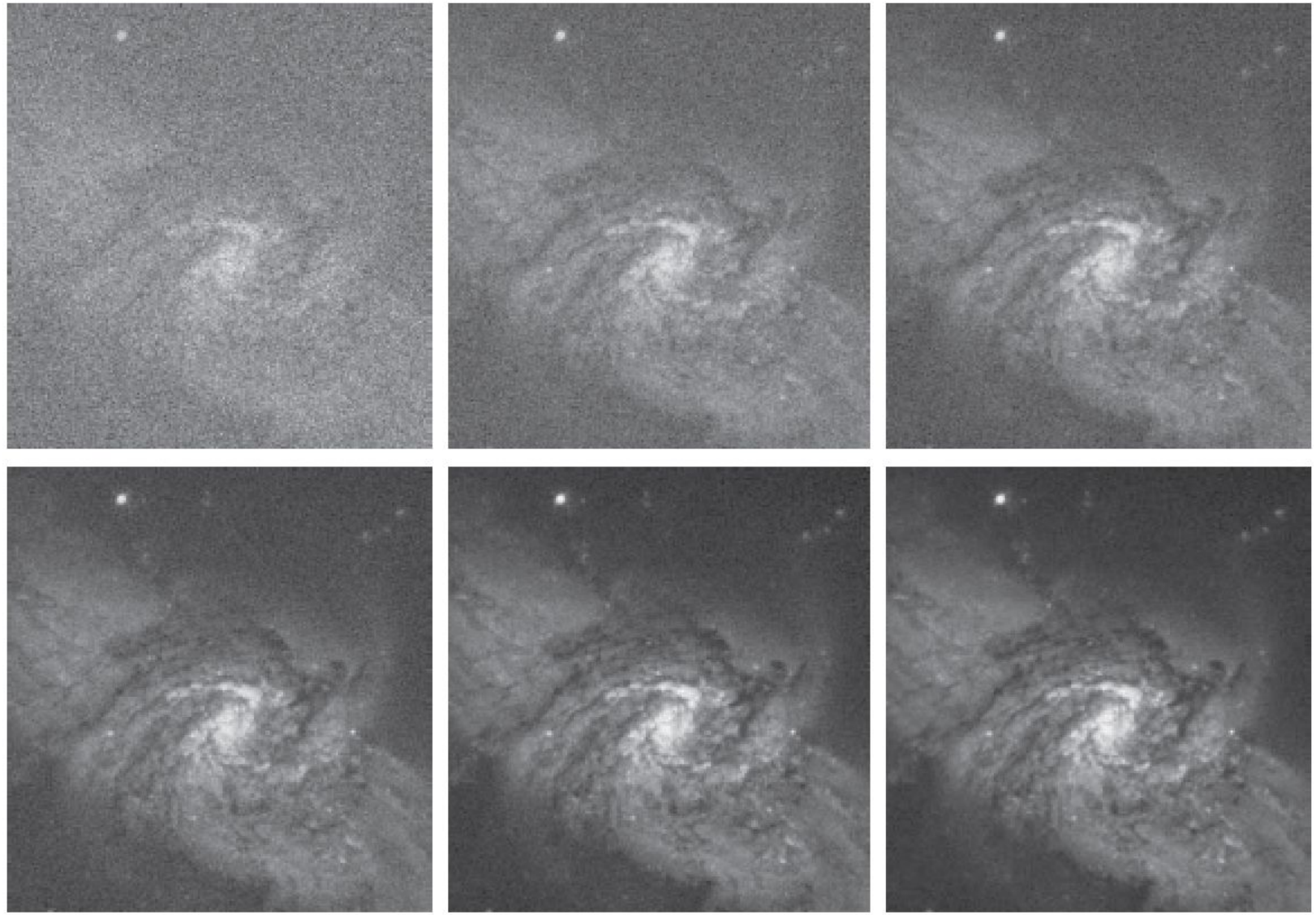
$$A = \begin{bmatrix} 2 & 1 & 0 & 3 \\ -1 & 0 & 2 & 4 \\ 4 & -2 & 7 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} -4 & 3 & 5 & 1 \\ 2 & 2 & 0 & -1 \\ 3 & 2 & -4 & 5 \end{bmatrix},$$

- Then

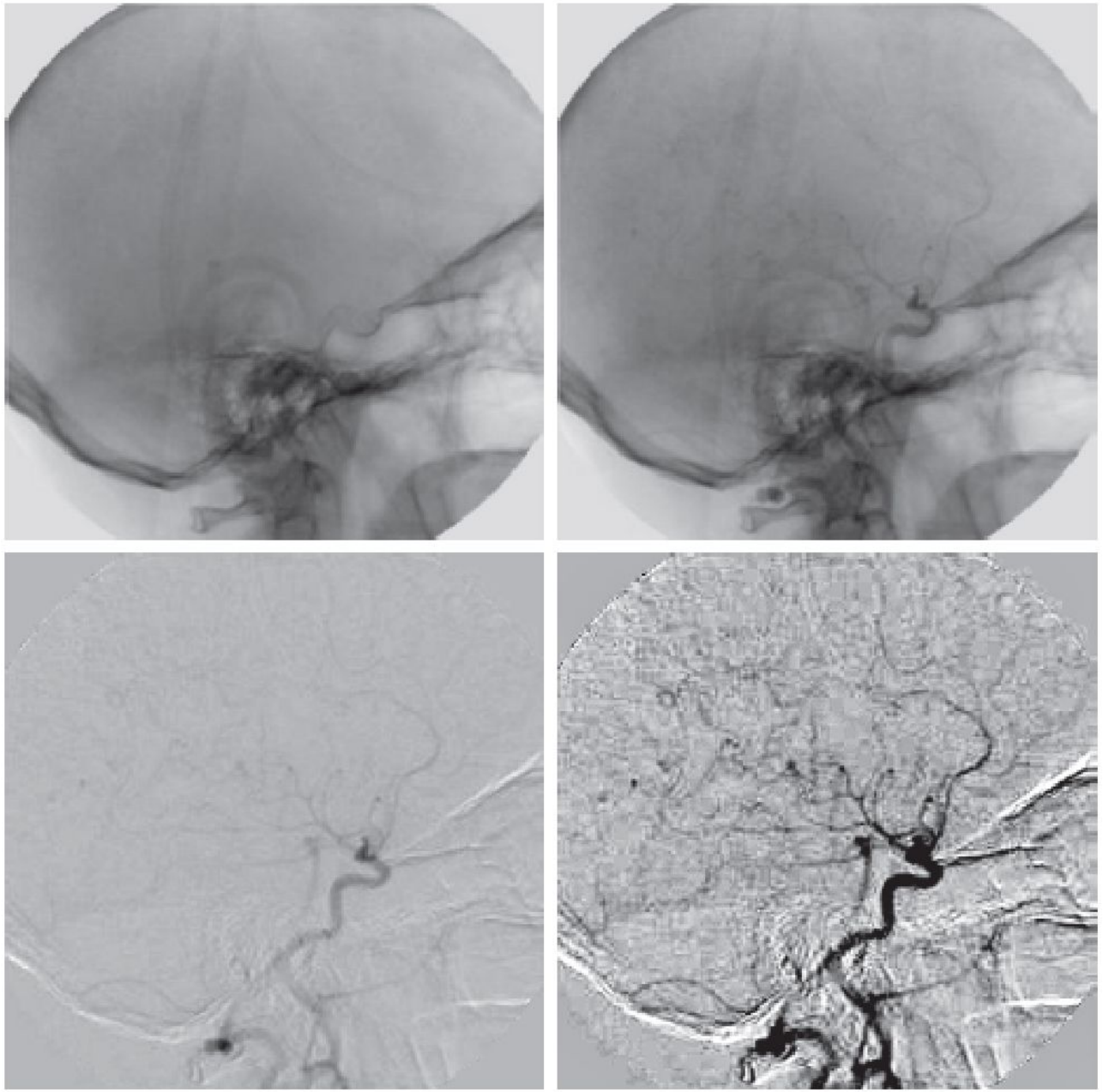
$$A + B = \begin{bmatrix} -2 & 4 & 5 & 4 \\ 1 & 2 & 2 & 3 \\ 7 & 0 & 3 & 5 \end{bmatrix}, \quad A - B = \begin{bmatrix} 6 & -2 & -5 & 2 \\ -3 & -2 & 2 & 5 \\ 1 & -4 & 11 & -5 \end{bmatrix}$$

Averaging : Averaging is an arithmetic operation used in image smoothing by replacing each pixel's intensity with the average of its neighboring pixels. This process reduces noise, smoothens edges, and minimizes variations in the image.

**ADDITION**



**AVERAGING**



**SUBTRACTION**

**TO ADJUST BRIGHTNESS OR TO MASK LIKE TO IDENTIFY FILLING**

**multiplication by 2**

**PRODUCT**

origin
al

shade
pattern

shade
correction

**DIVISION**

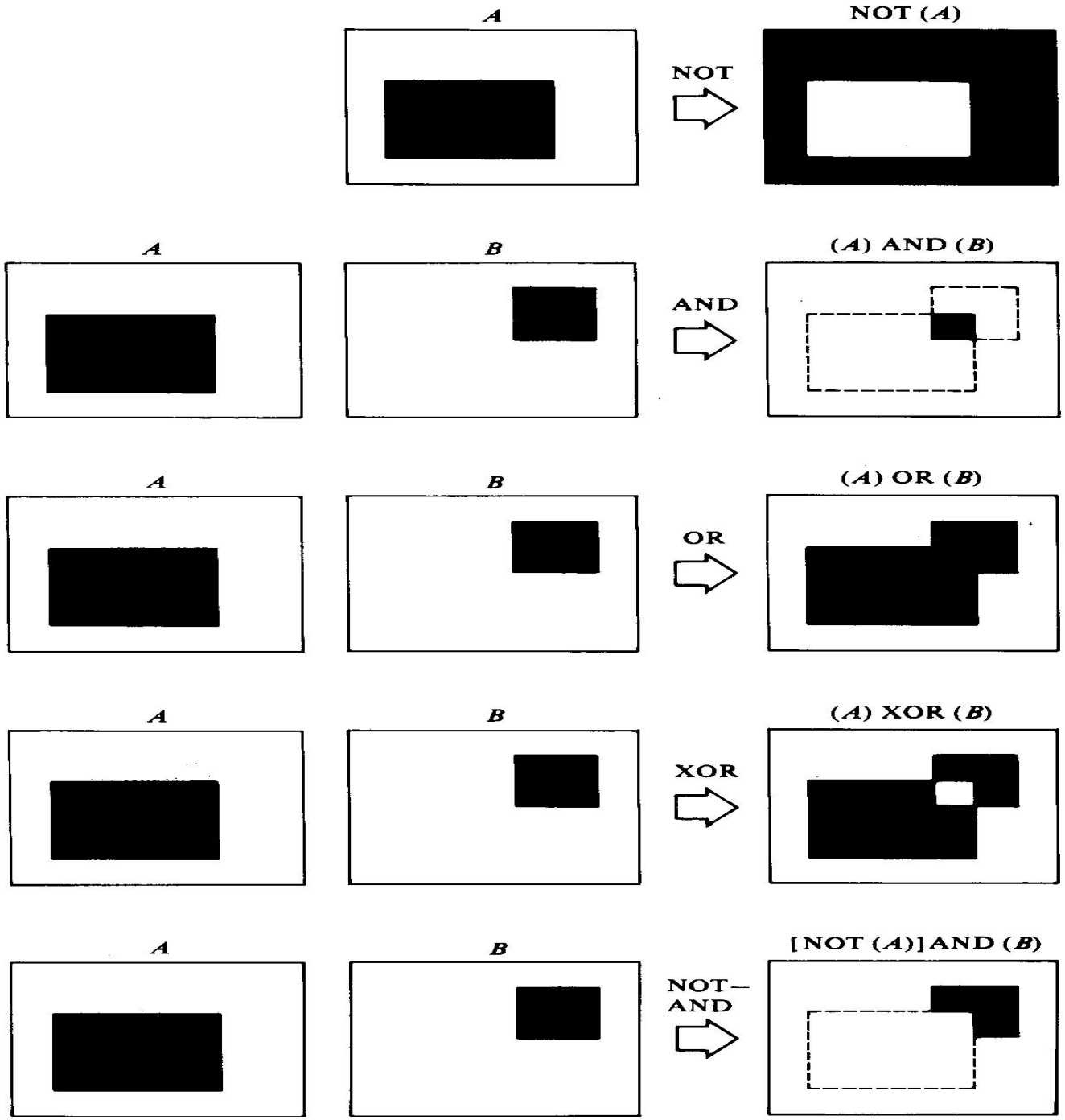
# Logical Operations

Basic tools of binary image processing where they are used for such tasks as masking, feature detection, and shape analysis

| | | |
|---|---|---|
| **And:** | p AND q | $p \bullet q$ |
| **Or:** | p OR q | $p + q$ |
| **Complement:** | NOT q | $q$ |

-

# Logical Operations Examples

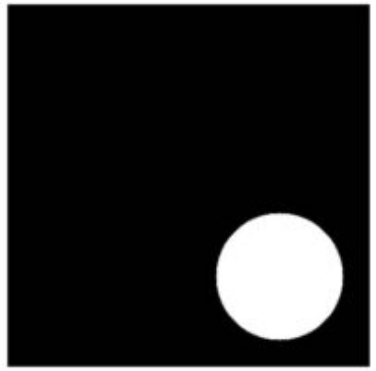


cv2.bitwise_and(A,B)

cv2.bitwise_or(A,B)

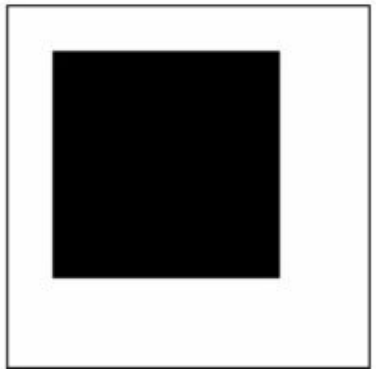
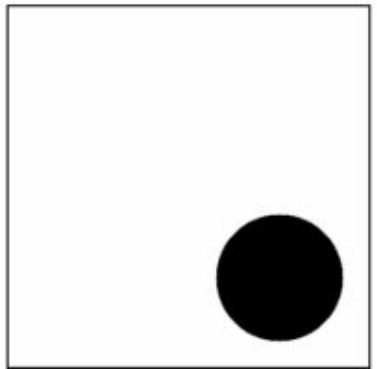
A

B



cv2.bitwise_xor(A,B)

cv2.bitwise_not(A)



cv2.bitwise_not(B)